CSI370 Computer Architecture

Software Project: Verte Compiler - Assembly Refactor

Andy Zheng
Champlain College
andy.zheng@mymail.champlain.edu

December 7, 2025

1 Project Overview

1.1 Project Description

This project aims to refactor the existing code generation backend of the `vertec` compiler. `vertec` is a compiler built for the `Verte` programming language which beforehand utilized `LLVM` for machine code generation. With the implemented project, the `LLVM` backend was replaced with a custom `x86-64 Assembly` backend. This new backend directly translates the abstract syntax tree (`AST`) created by the frontend layer into `x86-64 Assembly` instructions through several phases: **IR generation**, **register allocation**, **instruction selection**, and finally, **assembly generation & compilation**.

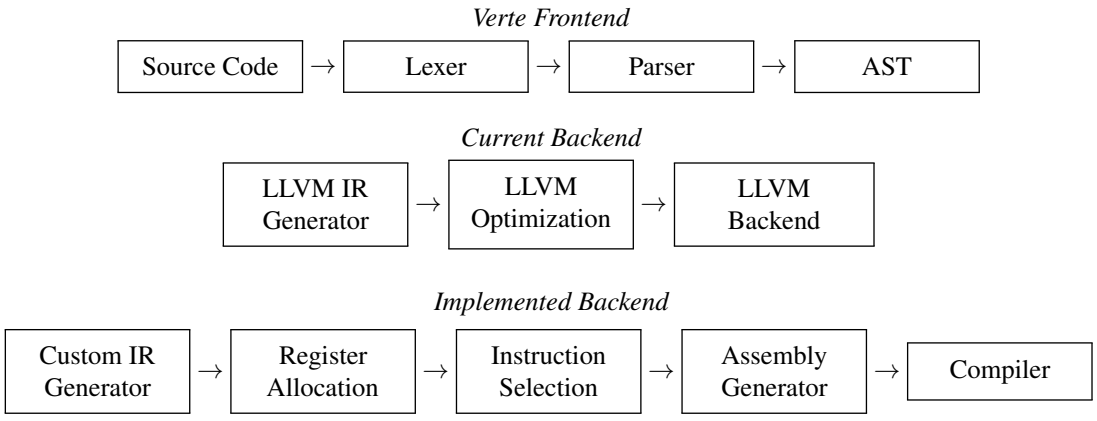

Figure 1: Verte Compiler Architecture with Proposed Backend

1.2 Motivation

The motivation behind this project comes from both educational and technical reasons. Firstly, I believe that building a custom assembly backend will provide a deeper understanding of low-level code generation, specifically a lot of theory can be read, but actually applying them is a different story; so I intend to see this as a checkpoint to verify my understanding of compiler design. Besides this, I also feel like it would be a good idea to explore alternatives to `LLVM` simply due to it's complexity and steep learning curve. Building a custom backend will alleviate this, and as a result of this, code generation will be more tailor-made, simplified, and understandable in contexts of maintaining the `Verte` programming language.

1.3 Approach and Methodology

The implementation of this refactor was carried out incrementally over several phases as mentioned in the project description. It first started with the analysis of the code base, and it's integration with the existing `LLVM` backend, this was done to best understand how to replace it without cascading changes through the existing code base for `Verte`. Following that, work was done incrementally based on the proposed backend outlined in Figure 1, starting with intermediate representation (`IR`), to register allocation, instruction selection, and finally assembly generation and compilation. Whilst development was ongoing, unit tests were also added to cover critical components in the `x86-64 Assembly` compilation pipeline. This was done in order to ensure that development was technically correct, verifiable, and acceptable.

2 Implementation

2.1 Intermediate Representation

Like mentioned earlier, the first phase of the refactor involved designing and implementing a custom `IR` to replace `LLVM`'s `IR` layer. This custom `IR` follows the Three-Address Code (`3AC`) format [1] shown in Equation 2, over Static Single Assignment (`SSA`) format, which is what `LLVM` uses. This choice was made because `SSA` requires a ϕ function, the ϕ function can be thought of as a special merge operator in `SSA` that handles control flow convergence. A general form of a ϕ function is show in Equation 1.

$$x_3 = \phi(B_1 : x_1, B_2 : x_2) \tag{1}$$

Equation 1: Phi-function Example

Where B_1 and B_2 are predecessor basic blocks, x_1 is the value of variable x defined in block B_1, x_2 is the value of x defined in block B_2, and x_3 is the merged result that takes the value of x_1 if control flow came from B_1, or x_2 if it came from B_2. While `SSA` does enable certain optimizations for complex programs, it introduces additional complexity to maintain the single-assignment property. For the purposes of this refactor, the simpler `3AC` format was deemed sufficient for `Verte` programs.

The general form for 3AC can be represented as $x = y \ \texttt{op} \ z$, where op represents the operator. At most, each instruction contains only three operands and no more. For example, a = b + c * d becomes:

$$\begin{aligned} t_1 &= c \times d \\ a &= b + t_1 \end{aligned} \tag{2}$$

Equation 2: Three-Address Code Example

In practice, the vertec compiler generates t_1 as a virtual register. The IR generator can assign an unlimited amount of virtual registers, which will then later be translated to physical registers through register allocation.

2.1.1 Operand Types and Type Information

In the IR implementation, we have five distinct operand types: **virtual register**, **immediates**, **floating-point immediates**, **globals**, and finally **stack slots**. Each operand carries type information within it through the structure, TypeInfo rather than inferring types from context. This enables early error detection during the IR generation phase, simplifies instruction selection by direct types, and facilitates automatic insertion of conversion instructions when mixing integer and floating-point arithmetic. Additionally to ensure the type-safe property of operands, the code base exposes factory methods shown in Listing 1 to construct these operands.

Listing 1: Operand Creation with Type Information

```
auto dest = ir::Operand::vreg(
    vregID,
    types::TypeInfo(types::TypeInfo::DataType::INTEGER)
);


auto foo = ir::Operand::imm(1);     // Implicit type.
auto bar = ir::Operand::fimm(1.0); // Implicit type.
auto baz = ir::Operand::global(".str0");
auto qux = ir::Operand::stackSlot(-8, intType);
```

2.1.2 Instruction Set

In the IR implementation we also define the instruction set able to be used by the compiler. The instruction set covers arithmetic for both integer and floating-point types (ADD, SUB, MUL, DIV, FADD, FSUB, ...), comparisons (ICMP_LT, FCMP_EQ, ...), type conversions (ITOF, FTOI), memory operations (LOAD, STORE), and control flow (BR, CONDBR, CALL, RET). These instructions are organized into basic blocks, of which only have a single entry and exit point. A basic block executes atomically, meaning if the first instruction runs, all instructions run in sequence. This property makes dataflow analysis much simpler as we can analyze the block as a unit.

2.1.3 Control Flow Graphs

Basic blocks are connected via predecessor and successor edges to form control flow graphs (`CFG`). These control flow graphs are essential for the register allocation's liveness analysis. Consider the `Verte` conditional program shown in Listing 2.

Listing 2: Verte Conditional Example

```
fun main(): @int do
    let x: @int = 1;
    let y: @int = 2;

    match where
        when x == y do
            return 0;
        end

        else -> return 1;
    end
end
```

With this example `Verte` program, we can visually translate this into a `CFG` shown in Figure 2, illustrating how control begins in block B_1, evaluates the condition, then takes one of two paths through B_2 or B_3, with both paths eventually converging at the exit.

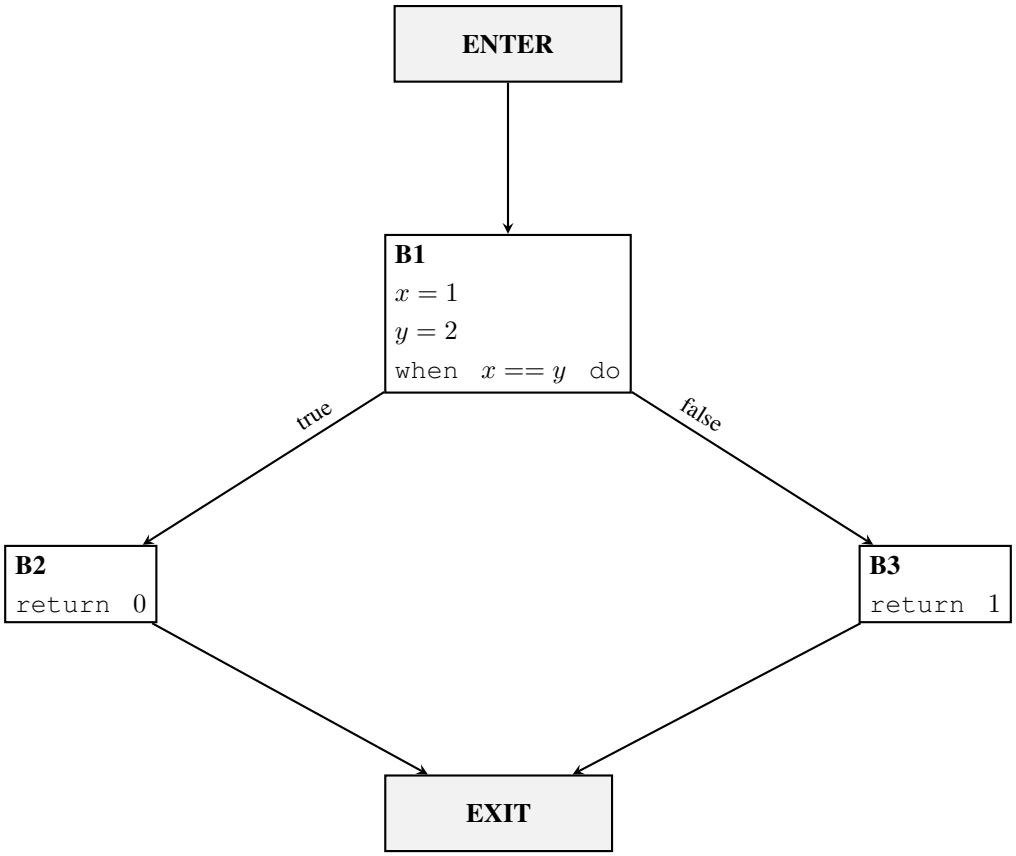

Figure 2: Control Flow Graph for Conditional Statement

The algorithm to construct CFGs in order to establish these predecessor and successor relationships is outlined in Algorithm 1.

Algorithm 1 Control Flow Graph Construction

```
1:  procedure BUILDCFGLINKS(func)
2:      for each block in func.getBlocks() do
3:          terminator ← last instruction in block
4:          if terminator is branch then
5:              target ← branch target block
6:              link block → target
7:          else if terminator is conditional branch then
8:              trueTarget ← true branch target
9:              falseTarget ← false branch target
10:             link block → trueTarget
11:             link block → falseTarget
12:         end if
13:     end for
14: end procedure
```

2.1.4 AST to IR Translation

To actually construct the `IR`, we use the `Codegen` class which implements the visitor pattern in order to traverse the `AST`, generating `IR` for each `AST` node type. For example, binary operations translate as shown in Listing 3.

Listing 3: Binary Operation IR Generation

```
auto Codegen::visit(const BinaryNode &node) -> RetT {
        auto lhs = std::get<ir::Operand>(node.getLHS()->accept(*this));
        auto rhs = std::get<ir::Operand>(node.getRHS()->accept(*this));
        const std::string &op = node.getOp();

        // Allocate result register.
        auto result = allocateVReg(lhs.getTypeInfo());

        ir::Opcode opcode;

        // Map operator to opcode.
        if (op == "+")
            opcode = ir::Opcode::ADD;
        else if (op == "-")
            opcode = ir::Opcode::SUB;

        // ... Additional operators ...

        currentBlock->addInstruction(
            ir::Instruction(opcode, result, lhs, rhs)
        );

        return result;
}
```

2.2 Register Allocation

While the `IR` generator is allowed to assign unlimited virtual registers, real CPUs only provide limited registers. To solve this we use a register allocation map to pair virtual registers with their physical counterparts. However, in the case that there are more live virtual registers compared to physical ones, some must be spilled to memory. To do this, we use an implementation of Linear Scan Allocation [2], which allocates registers in a single linear pass over the program's live intervals.

2.2.1 Live Interval Computation

To use linear scan, we first need live intervals. A live interval spans from a virtual register's first definition to its last use as seen in Equation 3. If two intervals overlap temporally, their registers will be unable to share the same physical registers. The algorithm computes intervals by numbering all instructions sequentially and tracking each virtual register's definition and last use indices.

$$\text{interval}(v) = [\text{firstDef}(v), \text{lastUse}(v)] \tag{3}$$

Equation 3: Live Interval Definition

In terms of implementation, Listing 4 shows how this is done in practice.

Listing 4: Computing Live Intervals

```
std::vector<LiveInterval>
RegisterAllocator::computeLiveIntervals(const ir::Function &func) {
  std::unordered_map<uint32_t, size_t> firstDef, lastUse;
  std::unordered_map<uint32_t, bool> isFloat;

  // Number instructions and track usage.
  size_t position = 0;
  for (const auto &block : func.getBlocks()) {
    for (const auto &instr : block->getInstructions()) {
      // Track uses in operands.
      for (const auto &operand : instr.getOperands()) {
        if (operand.isVReg()) {
          lastUse[operand.getVRegID()] = position;
        }
      }

      // Track definitions in destination.
      if (instr.getDest().isVReg()) {
        uint32_t vreg = instr.getDest().getVRegID();
        if (!firstDef.count(vreg)) {
          firstDef[vreg] = position;
        }

        lastUse[vreg] = position;
        isFloat[vreg] = isFloatType(
          instr.getDest().getTypeInfo()
        );
      }

      position++;
    }
  }

  // Construct intervals from tracking data.
  std::vector<LiveInterval> intervals;
  for (auto& [vreg, start] : firstDef) {
    intervals.emplace_back(
        vreg, start, lastUse[vreg], isFloat[vreg]
    );
  }

  return intervals;
}
```

2.2.2 Linear Scan Algorithm

With the intervals computed and sorted by start position, the linear scan algorithm processes them in order. For each interval, it first expires any currently active intervals that have ended (freeing their registers), then it attempts to allocate a register from the appropriate free pool of registers (integer or floating-point). When no registers remain, the algorithm must spill to memory. Algorithm 2 outlines the complete process.

Algorithm 2 Linear Scan Register Allocation

```
1:  procedure LINEARSCAN(intervals)
2:      Initialize freeIntRegs ← {R10, ...}
3:      Initialize freeFloatRegs ← {XMM8, ...}
4:      Initialize active ← ∅
5:      for each interval i in sorted intervals do
6:          EXPIREOLDINTERVALS(i, active, freeIntRegs, freeFloatRegs)
7:          if i.isInt then
8:              freeRegs ← freeIntRegs
9:          else
10:             freeRegs ← freeFloatRegs
11:         end if
12:         if freeRegs ≠ ∅ then
13:             reg ← pop from freeRegs
14:             assign reg to i
15:             add i to active
16:             if reg is callee-saved then
17:                 track reg for prologue/epilogue
18:             end if
19:         else
20:             SPILLINTERVAL(i, active)
21:         end if
22:     end for
23: end procedure
```

When all registers are exhausted, spilling becomes necessary. Spilled registers receive stack slots at negative offsets from the frame pointer: the first spill gets `[rbp - 8]`, the second `[rbp - 16]`, and so on. The decision of which interval to spill is made by checking if the current interval ends before all active intervals (since it has the shortest remaining lifetime); otherwise, spill the active interval with the furthest endpoint (which would occupy a register for the longest time).

2.3 Instruction Selector

After IR generation and register allocation on that IR, in order to have a proper x86-64 Assembly program we must select x86-64 Assembly instructions to represent our IR in code. We do this through pattern matching, essentially each IR opcode maps to an assembly string. This assembly string is produced through dedicated selection functions for each type of IR instruction.

2.3.1 Two-Address Form Translation

x86-64 Assembly arithmetic instructions uses the Two-Address form, where one operand serves two roles, being both source and destination. The 3AC IR instructions $r_3 = r_1 + r_2$ cannot be directly translated to add r3, r1. The correct sequence requires an explicit copy shown in Listing 5.

Listing 5: Two-Address Form Conversion

```
mov r3, r1
add r3, r2
```

We apply this pattern to all binary operations to ensure we correctly translate our 3AC IR code to x86-64 Assembly code.

2.3.2 Function Calls

Function calls are done through compliance with the System V ABI. The calling convention denotes exact register usage, and implementation wise we must strictly follow it. A critical requirement is the stack pointer alignment. The ABI requires that RSP must be aligned to a 16-byte boundary before the CALL instruction is executed. Since CALL pushes an 8-byte return address, and our function prologue pushes RBP (8 bytes), we must be careful with calculation and tracking. The implementation in Listing 6 demonstrates how this was done.

Listing 6: Function Call Code Generation

```
void InstructionSelector::selectCall(
    const ir::Instruction &instr,
    const RegisterAllocator &allocator,
    std::vector<X86Instruction> &code
) {
    const auto &args = instr.getOperands();
    const std::string &funcName = instr.getFuncName();

    size_t intArgIndex = 0;
    size_t floatArgIndex = 0;
    size_t stackArgs = 0;

    // Calculate stack space needed for stack arguments.
    for (size_t i = 0; i < args.size(); ++i) {
```

```
        bool isFloat = isFloatType(args[i].getTypeInfo());
        if ((isFloat && floatArgIndex >= 8) ||
        (!isFloat && intArgIndex >= 6)) {
            stackArgs++;
        }

        isFloat ? floatArgIndex++ : intArgIndex++;
    }

    // Align stack to 16 bytes if needed.
    size_t stackSpace = stackArgs * 8;
    if (stackSpace % 16 != 0) {
        stackSpace += 8;
    }

    if (stackSpace > 0) {
        code.push_back(X86Instruction("sub", "rsp", std::to_string(
            stackSpace)));
    }

    // Reset indices for actual argument passing.
    intArgIndex = 0;
    floatArgIndex = 0;
    size_t stackOffset = 0;

    // Pass arguments.
    for (const auto &arg : args) {
        std::string argStr = formatOperand(arg, allocator);
        bool isFloat = isFloatType(arg.getTypeInfo());

        if (isFloat && floatArgIndex < 8) {
            std::string xmmReg =
                getRegName64(FLOAT_ARG_REGS[floatArgIndex++]);
            code.push_back(X86Instruction("movsd", xmmReg, argStr));
        }

        else if (!isFloat && intArgIndex < 6) {
            std::string intReg =
                getRegName64(INT_ARG_REGS[intArgIndex++]);
            code.push_back(X86Instruction("mov", intReg, argStr));
        }

        else {

            std::string stackLoc =
                "[rsp + " + std::to_string(stackOffset) + "]";

            std::string instr = isFloat ? "movsd" : "mov";
            code.push_back(X86Instruction(instr, stackLoc, argStr));
```

```
                stackOffset += 8;
            }
        }

        // Call the function.
        code.push_back(X86Instruction("call", funcName));

        // Clean up stack space.
        if (stackSpace > 0) {
            code.push_back(
                X86Instruction("add", "rsp", std::to_string(stackSpace))
            );
        }

        // Move return value to destination register.
        if (instr.getDest().isVReg()) {
            std::string destStr = formatOperand(instr.getDest(), allocator);
            bool isFloat = isFloatType(instr.getDest().getTypeInfo());
            std::string retReg = isFloat ? "xmm0" : "rax";

            if (destStr != retReg) {
                std::string instr = isFloat ? "movsd" : "mov";
                code.push_back(X86Instruction(instr, destStr, retReg));
            }
        }
    }
```

2.4 Generation and Compilation

So far, what has been described generally is the individual components of the compiler pipeline shown in Figure 3. We are now at the final stage of that process, in this phase `NASM x86-64 Assembly` files are created from the selected instructions through the assembly generator, then external tools are called to assemble and link the results into executable programs.

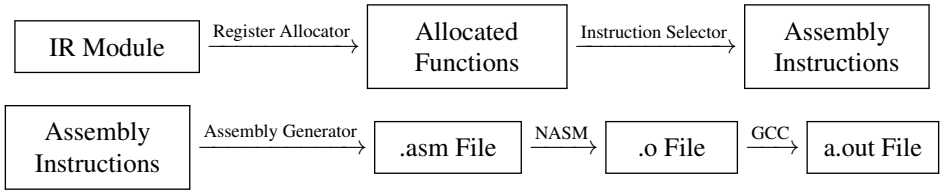

Figure 3: Compilation Pipeline Stages

2.4.1 Assembly File Generation

The assembly generator starts by emitting the data section which contains initialized read-only data, such as floating-point constants and string literals. After this, the bss section is emitted and is reserved for

uninitialized data, although currently unused in our implementation as globals are required to be constants in `Verte`. Then finally, the text section is emitted which contains the executable code for all functions.

Each function in the assembly code follows the standard structure. The prologue establishes the stack frame by pushing the base pointer, setting up `RBP`, allocating stack space, and saving callee-saved registers that were used by the register allocator. The function body contains the selected instructions translated to assembly code. The epilogue restores callee-saved registers, deallocates the stack frame, and returns control to the caller. A minimal example of this structure is shown in Listing 7.

Listing 7: Assembly Function Structure

```
section .text
    global main

main:
    push rbp      ; Save old base pointer.
    mov rbp, rsp  ; Set up new frame.
    sub rsp, 32   ; Allocate stack space.

    ; Function body.

    mov rsp, rbp  ; Deallocate stack.
    pop rbp       ; Restore base pointer.
    ret
```

2.4.2 Compilation Pipeline

After assembly generation, the compilation pipeline calls on external tools to produce an executable from the generated `x86-64 Assembly` file. It first determines the file paths for the assembly and object file, using the base name from the output path. The pipeline then executes sequentially the three stages shown in Figure 3

The first stage calls the assembly generator's `writeToFile` method, which generates the complete `x86-64 Assembly` file and writes it to disk. This stage combines all previously computed information, that being the allocated registers, selected instructions, and constants table into a single assembly file. If this stage fails, the pipeline terminates immediately with an error.

The second stage calls `NASM` through the `runAssembler` method. The command executed is `nasm -f elf64 -o output.o output.asm`, which assembles the generated assembly file into an `ELF64` object file. If assembly fails, intermediate files are cleaned up based on compiler options passed by flags to `vertec`.

The final stage calls `GCC` through the `runLinker` method with the command `gcc -o output output.o -no-pie -z noexecstack`. The `-no-pie` flag disables position-independent executable generation, simplifying address calculations in our generated code. The `-z noexecstack` flag marks the stack as non-executable, a security feature that prevents code execution from stack memory. If linking

succeeds and the compiler options do not request preservation of intermediate files, the pipeline automatically removes the temporary assembly and object files, leaving only the final executable.

3 Challenges

3.1 Parameter Register Allocation Strategy

Function parameter handling was not so trivial. In the System V ABI calling convention, parameters arrive in designated argument registers: `RDI`, `RSI`, `RDX`, `RCX`, `R8`, and `R9` for integers, and `XMM0` through `XMM7` for floating-point values. However, several of these registers are caller-saved, meaning they can be freely overwritten by called functions without preservation. This creates a problem when a parameter's value must survive across function calls within the body.

The initial register allocation implementation treated parameter virtual registers like any other virtual register, often assigning them to caller-saved registers like `R10` or `R11`. When the function subsequently called another function, these parameter values would be clobbered, producing incorrect results. The issue manifested subtly in recursive functions or functions with multiple internal calls that needed to reference parameters after making those calls.

The solution involved biasing the register allocator to prefer callee-saved registers (`RBX`, `R12`, `R13`, `R14`, `R15`) for function parameters. These registers are preserved across function calls by convention, as any called function must save and restore them. By allocating parameters to callee-saved registers, their values naturally survive function calls without requiring explicit saves and restores. The trade off is that using callee-saved registers requires the prologue to save them and the epilogue to restore them, adding overhead. However, this cost is typically cheaper than repeatedly spilling and reloading parameter values around each function call.

4 Conclusion

This project successfully replaced LLVM with a custom x86-64 Assembly backend for the vertec compiler. The implementation demonstrates fundamental compilation techniques: 3AC IR design, linear scan register allocation, pattern-based instruction selection, and ABI compliant assembly generation. It is fair to say that the educational value of this project has far exceeded initial expectations.

For example, designing the IR showed how intermediate representations must balance abstraction with practical code generation concerns. Not to mention each decision about operand types, instruction formats, and control flow structures influenced every subsequent phase, demonstrating how foundational choices cascade throughout a compiler. On the other hand implementing register allocation brought forth the constraints of physical hardware into view. The naive and simple abstraction of unlimited virtual registers collides harshly with the reality that is finite physical registers, forcing decisions to be made on which values to keep in registers, and which to spill into memory. Beside this, instruction selection exposed just how wide the gap between high-level operations and actual machine code can be. Operations that seem simple in IR notation often translate to multiple x86-64 Assembly instructions that need to be carefully orchestrated, each with its own register requirements and side effects. This aspect was seen a lot while working with the ABI, and it definitely reinforced the idea that conventions exist for good reasons. Even small deviations from the expected calling conventions lead to subtle bugs that manifest only in specific circumstances, making them frustrating to diagnose.

The new backend, while certainly less optimized than LLVM, offers valuable experience and transparency in return. Every transformation is visible and understandable, making it possible to trace exactly how source code becomes machine code. This clarity proved invaluable during development and will continue to benefit future maintenance and enhancement. The code base now provides a solid foundation for further exploration, whether that involves adding optimization passes, experimenting with different register allocation strategies like graph coloring, or extending support to additional target architectures beyond x86-64 Assembly.

References

[1] Aho, A. V. (2007). 6.2 Three-Address Code. In *Compilers: Principles, Techniques & Tools*.

[2] Poletto, M., & Sarkar, V. (1999). Linear Scan Register Allocation. *ACM Transactions on Programming Languages and Systems Volume 21, Issue 5*.